

Exercise 3: Diagnose and Fix a Defect (Live) — 25 minutes

Scenario (student-facing)

(Trainers: a defect must be planted before attendees start this exercise — see [exercise_03_preparation.md](#).)

There is now a new branch of this repository. Run:

```
git checkout exercise-03
```

to catch up. This new version has caused a test to fail in the inventory service. We only have this defect report:

`InventoryServiceTest#totalShortageAcrossMultipleItemsIgnoresItemsAboveReorderLevel` is failing.

`InventoryService.totalReorderShortage` is supposed to sum how many units are needed across a list of items to bring every item up to its reorder level. Items that already have enough stock should not affect the total. With one item short by 8 units and another item 45 units above its reorder level, the method returns `-37` instead of `8`.

Prompt template

```
a test is failing in the inventory service.  
  
investigate the failure before changing code.  
please:  
1. run the smallest relevant test (InventoryServiceTest).  
2. identify the incorrect assumption or calculation in totalReorderShortage.  
3. explain the expected versus actual result.  
4. propose the smallest fix.  
  
do not edit files yet.  
do not inspect unrelated classes such as InventoryRepository or  
InventoryServer.
```

Then, after reviewing the diagnosis:

```
apply only the proposed fix.  
add or adjust a focused regression test if necessary.  
run the relevant test again and show the final diff summary.
```

What you should expect

- The bug: each item's contribution to the total is `reorderLevel - quantity` without clamping negative values (surplus items) to `0`. A correct implementation clamps each item's individual shortfall at `0` before summing. For example `Math.max(0, item.reorderLevel() - item.quantity())`.
- The fix is one line inside the loop.
- `mvn test -Dtest=InventoryServiceTest` should pass afterward.

Teaching points

- The agent should reproduce the problem (run the failing test) before proposing a fix, not just read the code and guess.
- A passing test afterward does not prove the diagnosis was correct, verify the agent's explanation of *why* the bug occurred against the actual code, not just that the assertion now succeeds.

- Distinguish symptom removal (hardcoding 8 or special-casing the two test items) from correcting the underlying rule (clamping every item's shortfall at zero, which fixes the calculation for any input).
- Models benefit from explicit expected-versus-actual values — the defect report above gives both -37 (actual) and 8 (expected) on purpose.